

Software Deployment

Software Deployment

- “The process of making software available for use by end-users”
- Deployment is the “final” step in the software development lifecycle, where the software is released to the end-users.
- This involves hosting the software on a server, configuring the environment, and making sure it's accessible to the users.



Software Deployment

- **Traditional software development lifecycle: planning, design, development, testing, deployment, maintenance**
- **Deployment's role: making software available for use, providing feedback loop for further development**
 - Deployment's impact on software quality:
 - Performance
 - Security
 - Maintainability
 - User experience

Critical factors to consider

- Ensures software availability and accessibility
- Affects user experience and satisfaction
- Impacts business operations and revenue
- Security and compliance considerations
- Continuous deployment and integration

Deployment

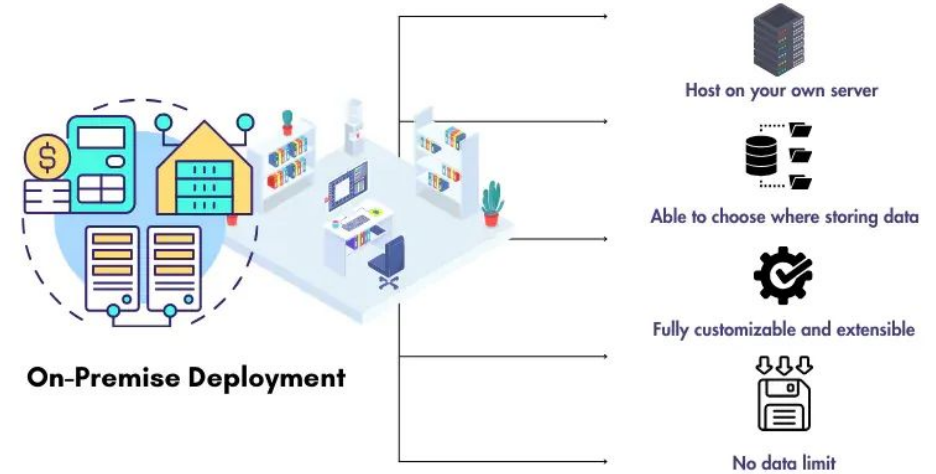
- Proper deployment is crucial for the success of any software development project.
- By understanding the importance of deployment and following best practices, you can ensure that your software is available, accessible, and secure for your end-users.

Types of Deployment

- On-premise deployment
- Cloud deployment
- Hybrid deployment
- Continuous deployment

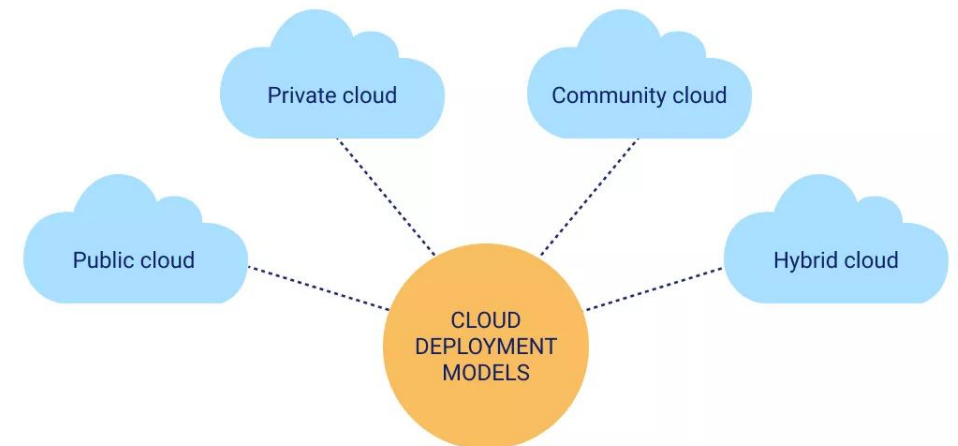
On-premise deployment

- On-premise deployment refers to the installation and operation of software on a company's **own** servers and infrastructure, rather than on a remote cloud server. In this model, the company is responsible for maintaining the hardware, software, and networking environment.



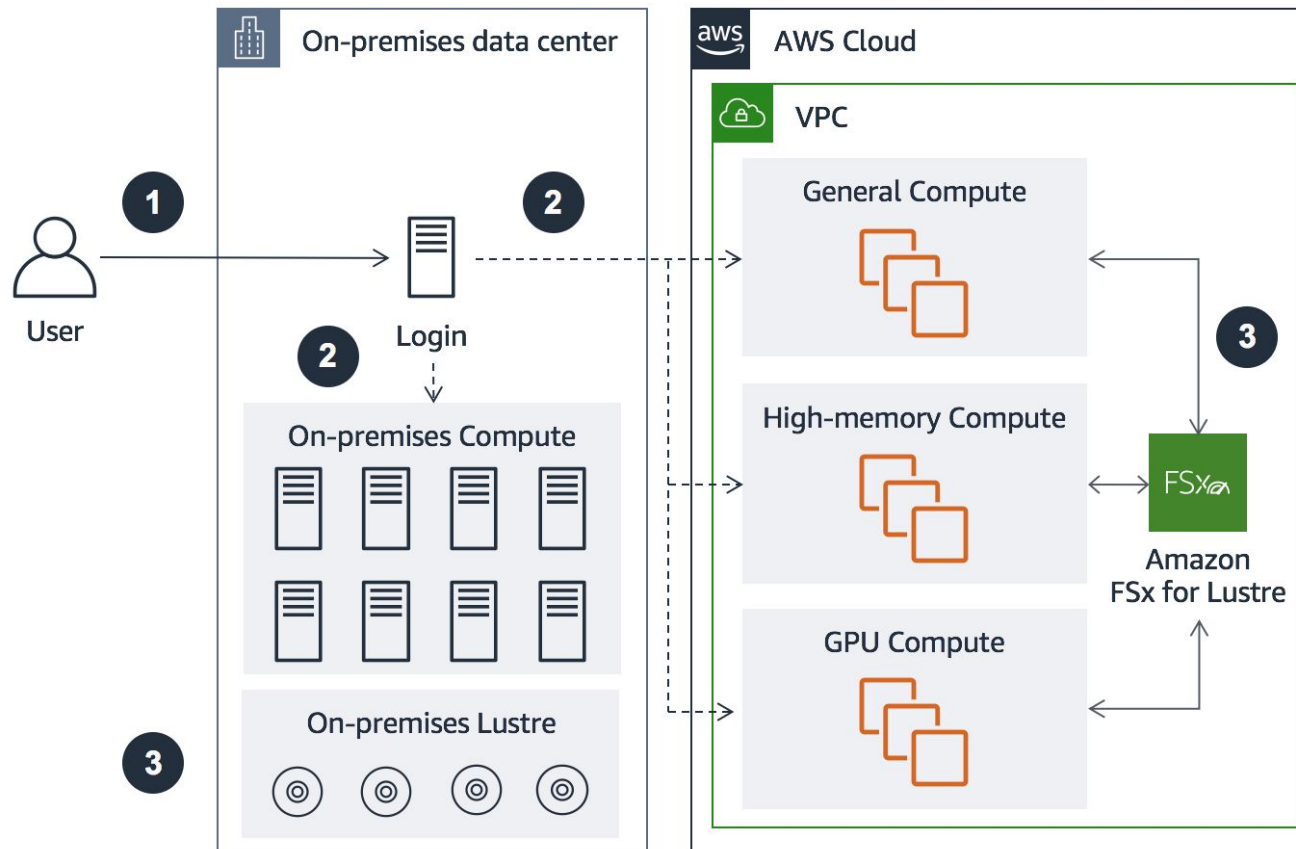
Cloud deployment

- Cloud deployment refers to the process of hosting applications, services, or infrastructure in a cloud environment—which means using **remote** servers accessed via the internet rather than hosting on local, on-premise hardware. These cloud resources are typically provided by third-party vendors like Amazon Web Services (AWS), Microsoft Azure, or Google Cloud Platform (GCP).



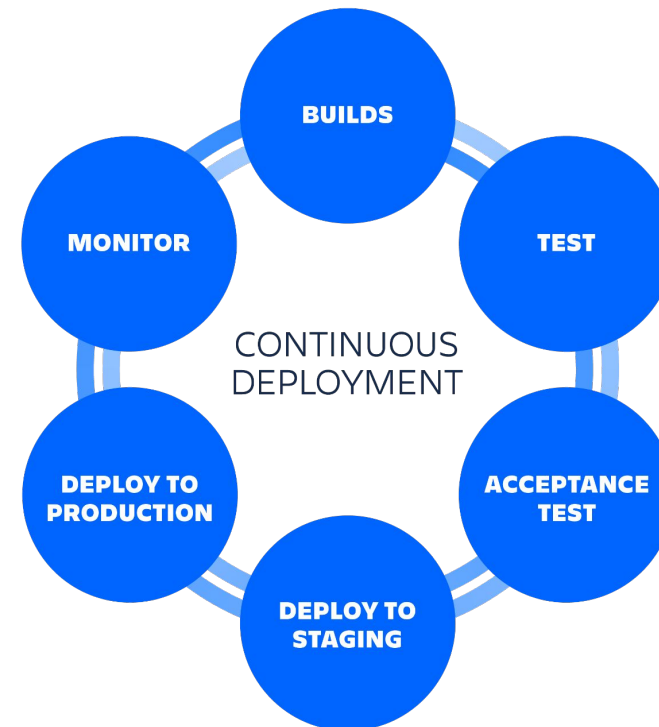
Hybrid deployment

- Hybrid deployment refers to a combination of on-premise infrastructure and cloud services.



Continuous deployment

- **Continuous Deployment (CD)** is an advanced software release practice where every change made to the codebase is automatically built, tested, and deployed to production without manual intervention.



Deployment Strategies

- Big Bang deployment
- Phased deployment
- Canary deployment

Big Bang deployment

Big Bang Deployment is a software deployment strategy where a new version or a significant update of a system is released **all at once**, typically to all users or environments at the same time.



Phased deployment

Phased Deployment, also known as **Incremental** Deployment or Rolling Deployment, is a deployment strategy where a new version of a system or software is gradually rolled out across multiple users, environments, or regions.

Canary deployment

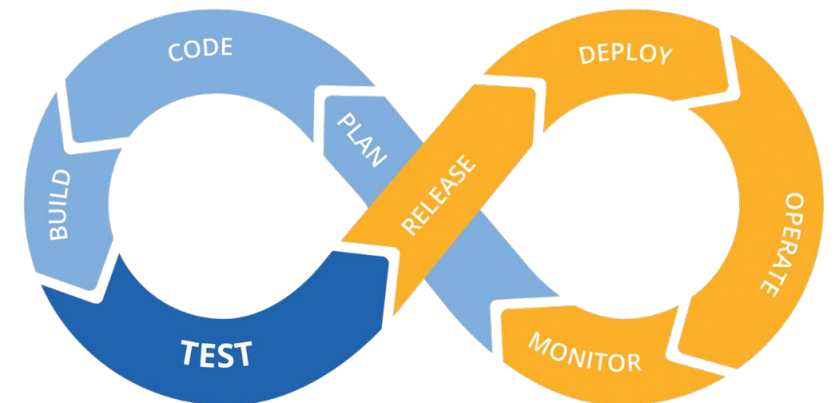
Canary Deployment is a deployment strategy where a new version of a software or system is first released to **a small subset of users** (the "canary" group) before gradually rolling it out to the rest of the user base.

Deployment Best Practices

- Automating deployment processes
- Testing deployments thoroughly before deploying to production
- Monitoring deployments for issues and performance
- Documenting deployment processes and procedures

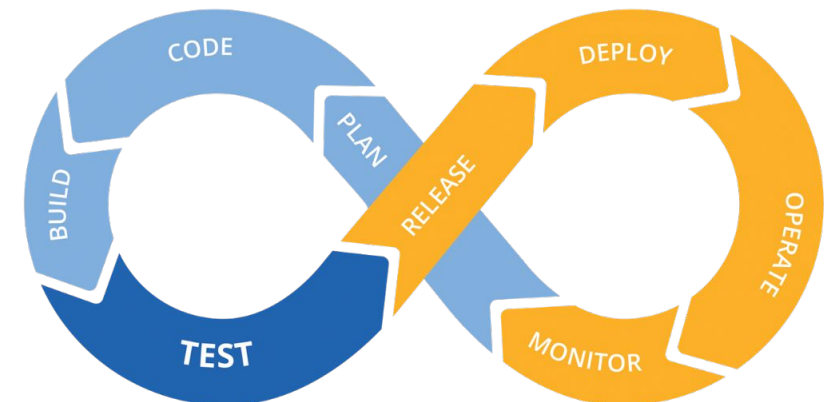
Tips and tricks for successful deployments

- Plan and test deployments in advance
- Use version control for deployment scripts
- Monitor and troubleshoot deployments
- Continuously improve and refine deployment processes
- Document deployment procedures



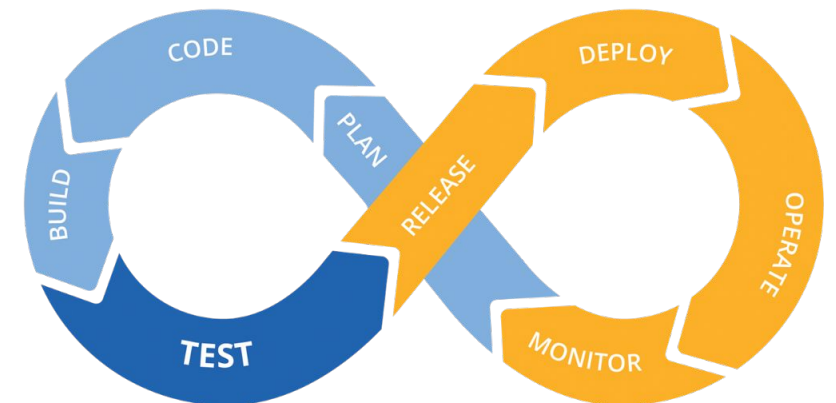
CI vs CD

- **Continuous Integration (CI):**
 - Merge code frequently
 - Run automated tests
- **Continuous Deployment (CD):**
 - Automatically release code to users
 - No manual steps



Benefits of CD

- **Faster delivery of features**
- **Reduced human error**
- **Immediate feedback from users**
- **Smaller, safer updates**

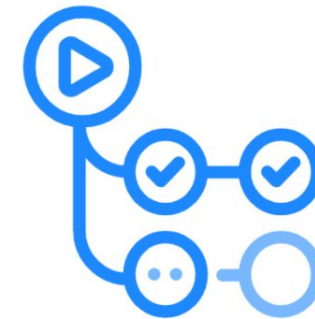


CD Tools Ecosystem

- **Typical tools used in CD:**
 - Containerization: Docker
 - Orchestration: Kubernetes
 - CI/CD Platforms:
 - GitHub Actions
 - GitLab CI/CD
 - Jenkins

Automated Packaging with GitHub Actions

- What is packaging?
- Why automate it?
- How GitHub Actions handles it



GitHub Actions

What is Packaging?

Packaging is the process of:

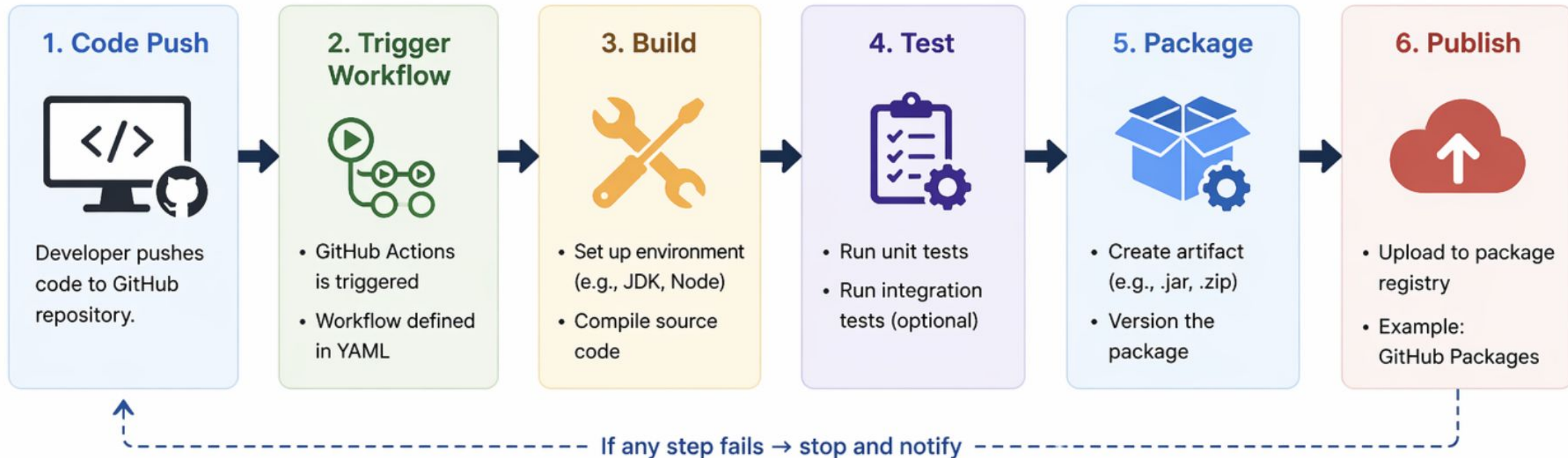
- **Compiling source code**
- **Bundling it into a distributable format**
 - .jar (Java)
 - .npm package
 - .zip

Why Automate Packaging?

- **Without automation:**
 - Manual builds
 - Inconsistent versions
 - Human errors

- **With automation using GitHub Actions:**
 - Consistent builds
 - Faster releases
 - Reproducible artifacts

Packaging Workflow



Example: Java Packaging with Maven

Using Apache Maven + GitHub Actions:

- name: Build and Package

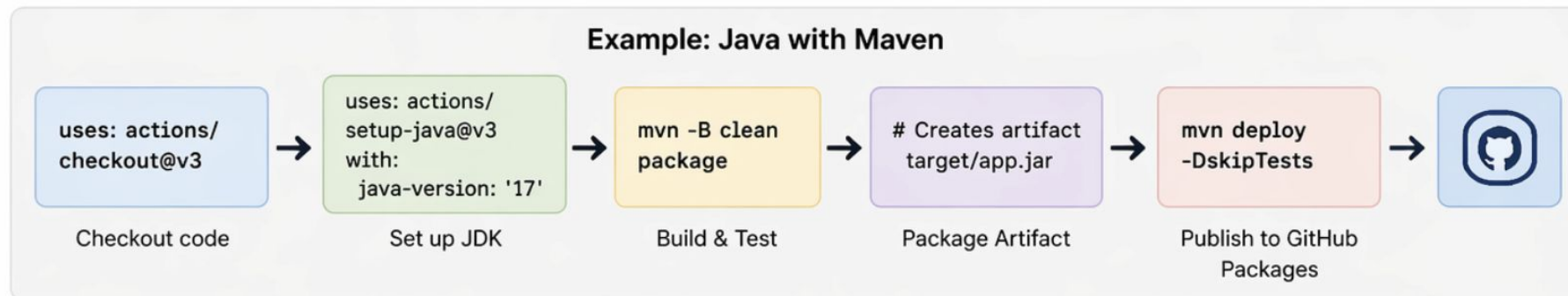
```
run: mvn -B package --file pom.xml
```

- name: Publish to GitHub Packages

```
run: mvn deploy -DskipTests
```

env:

```
GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }
```



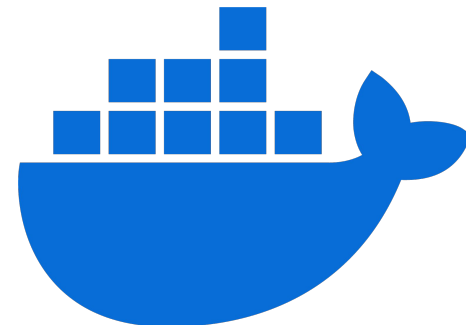
What Happens in This Workflow?

- Code is checked out
- Java environment is set up
- Maven builds the project
- Artifact (.jar) is created
- Artifact is uploaded to GitHub Packages

Docker

Docker is an open-source platform that enables developers to automate the deployment, scaling, and management of applications within lightweight, portable containers.

Docker allows developers to package an application and its dependencies into a lightweight, portable container.



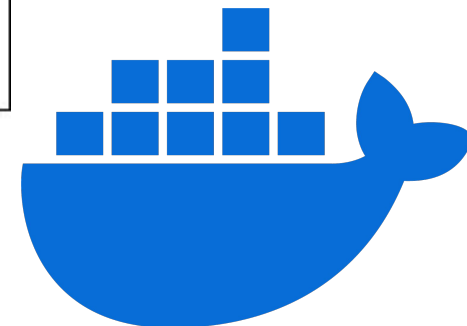
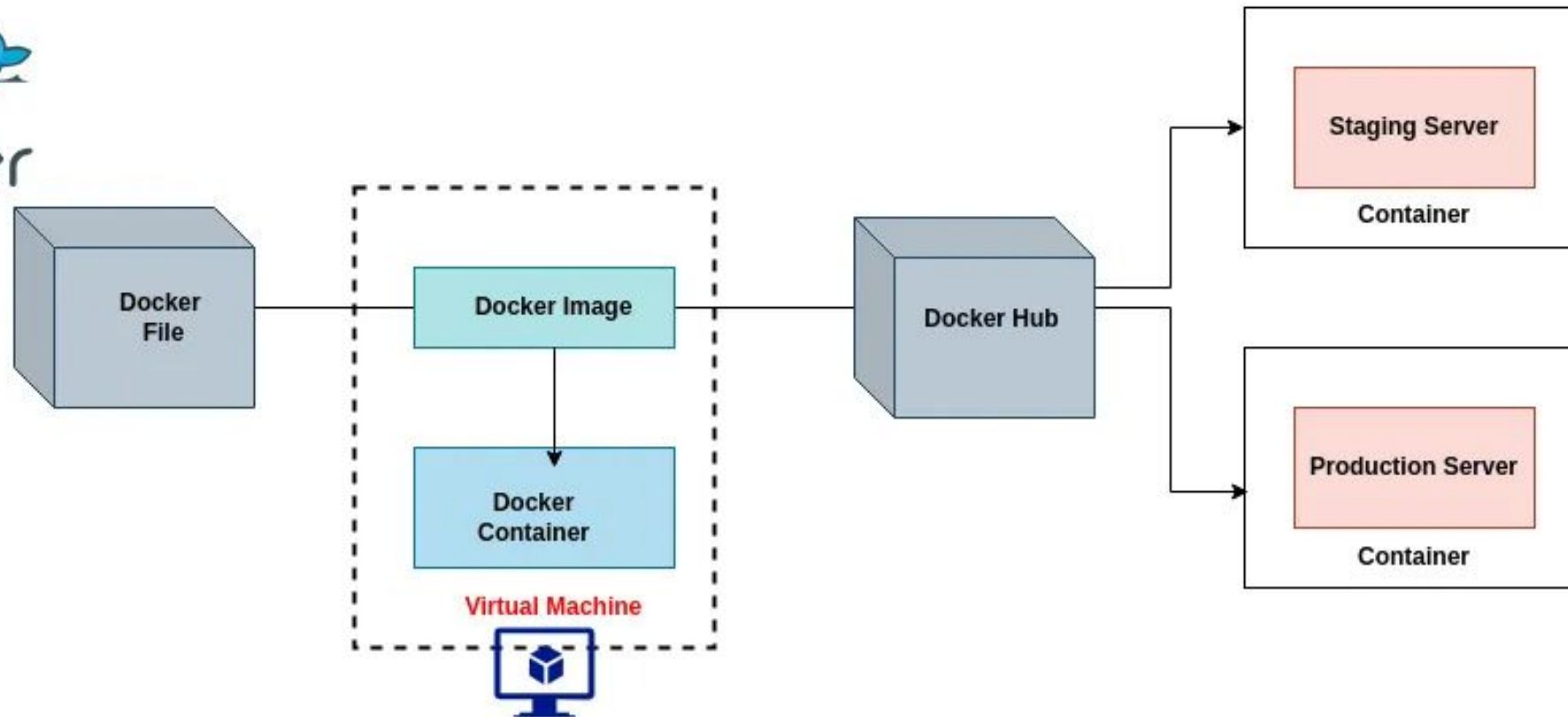
Role of Docker in CD

Docker standardizes how applications are packaged and delivered in CD pipelines:

- Build Docker image after successful tests
- Push image to container registry (e.g., Docker Hub)
- Provide a ready-to-deploy artifact

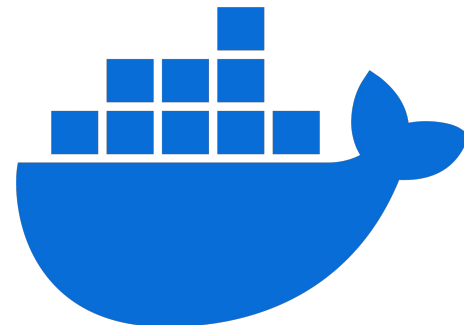
Docker

Docker Workflow



Docker

- Dockerfile:
 - A Dockerfile is a text document that contains the instructions needed to build a Docker image.
 - It defines the steps to install dependencies, set environment variables, copy files, and run commands inside the container.



Docker

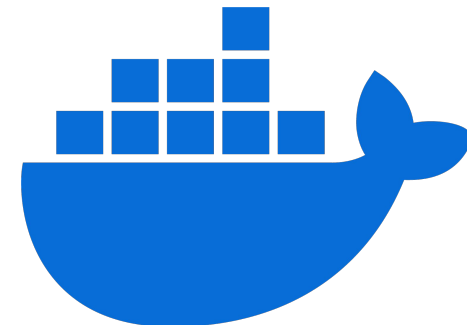
■ Image:

- A Docker image is a read-only template used to create containers.
- A Docker image is a snapshot of a complete file system (binary package), which includes: application code, dependencies and libraries, OS-level components, metadata and startup instructions.
- Once an image is created, it can be used to create multiple containers, ensuring that the application runs the same way in any environment.

Docker

■ Container:

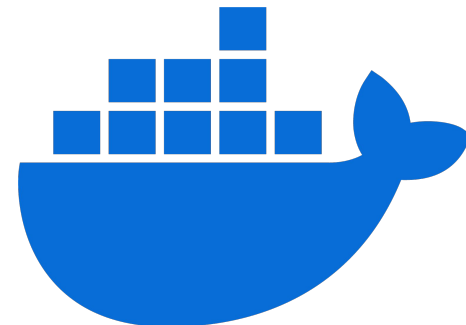
- A container is a lightweight, stand-alone, and executable software package that includes everything needed to run a piece of software, including the code, runtime, system tools, libraries, and settings.
- Containers are isolated from each other and the host system, providing consistent environments for applications to run, regardless of where they are deployed.



Docker

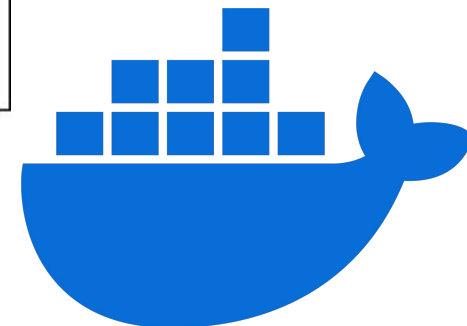
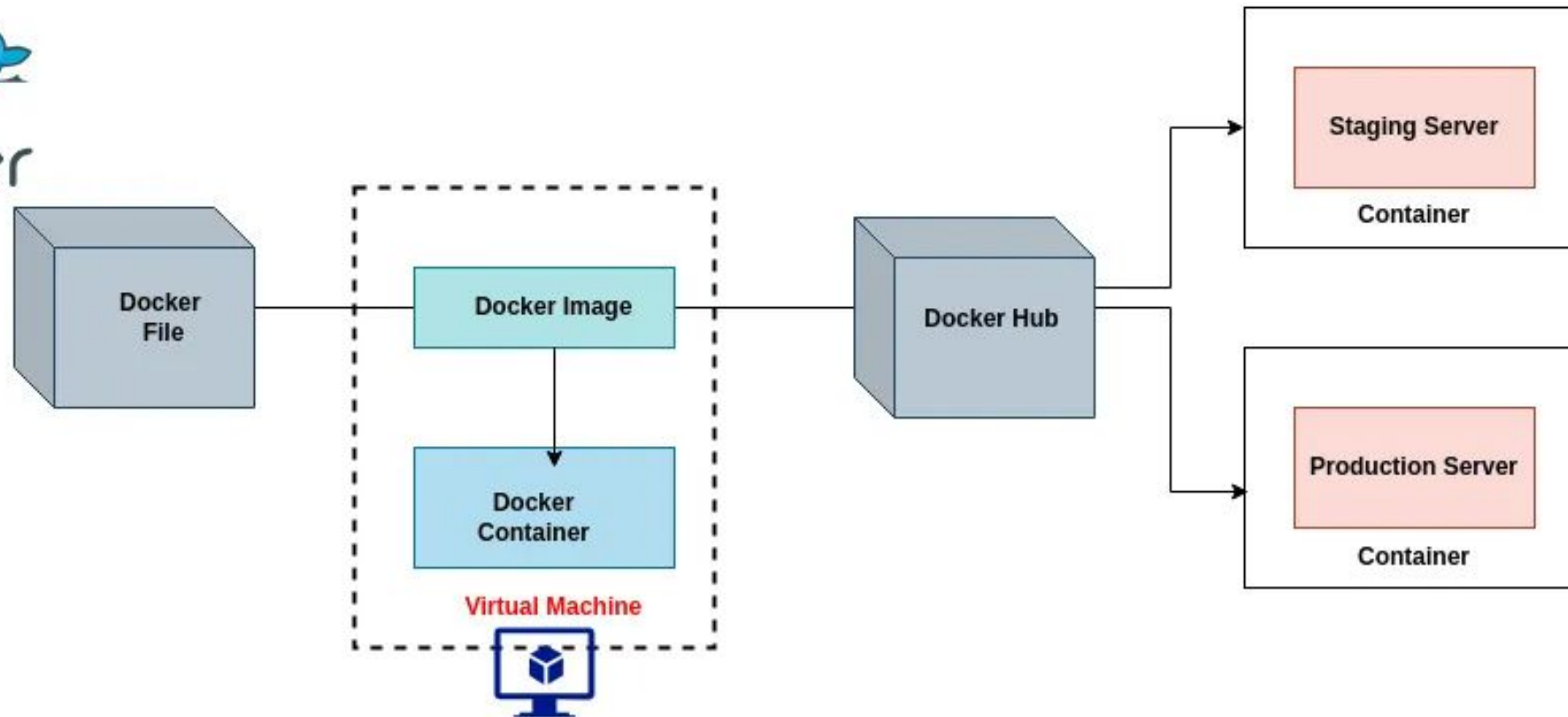
■ Docker Hub:

- Docker Hub is a cloud-based repository where Docker images are stored and shared. It contains official images (e.g., for popular databases or programming environments) and user-contributed images.
- Users can push their custom images to Docker Hub and pull images from the repository to deploy them locally.



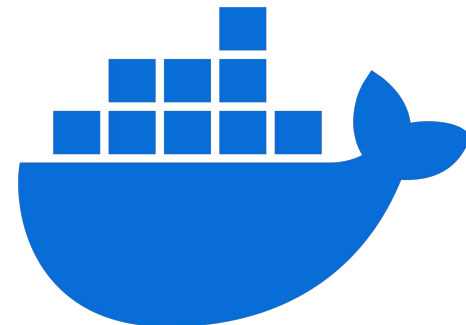
Docker

Docker Workflow



Docker

- Benefits:
 - Lightweight and portable containers
 - Faster development and deployment cycles
 - Isolation and security
 - Efficient resource utilization
 - Collaboration and reuse of images and containers



Kubernetes

Kubernetes is an open-source platform for automating the deployment, scaling, and management of containerized applications. It helps you coordinate containers across a cluster of machines, ensuring high availability, scalability, and reliability.

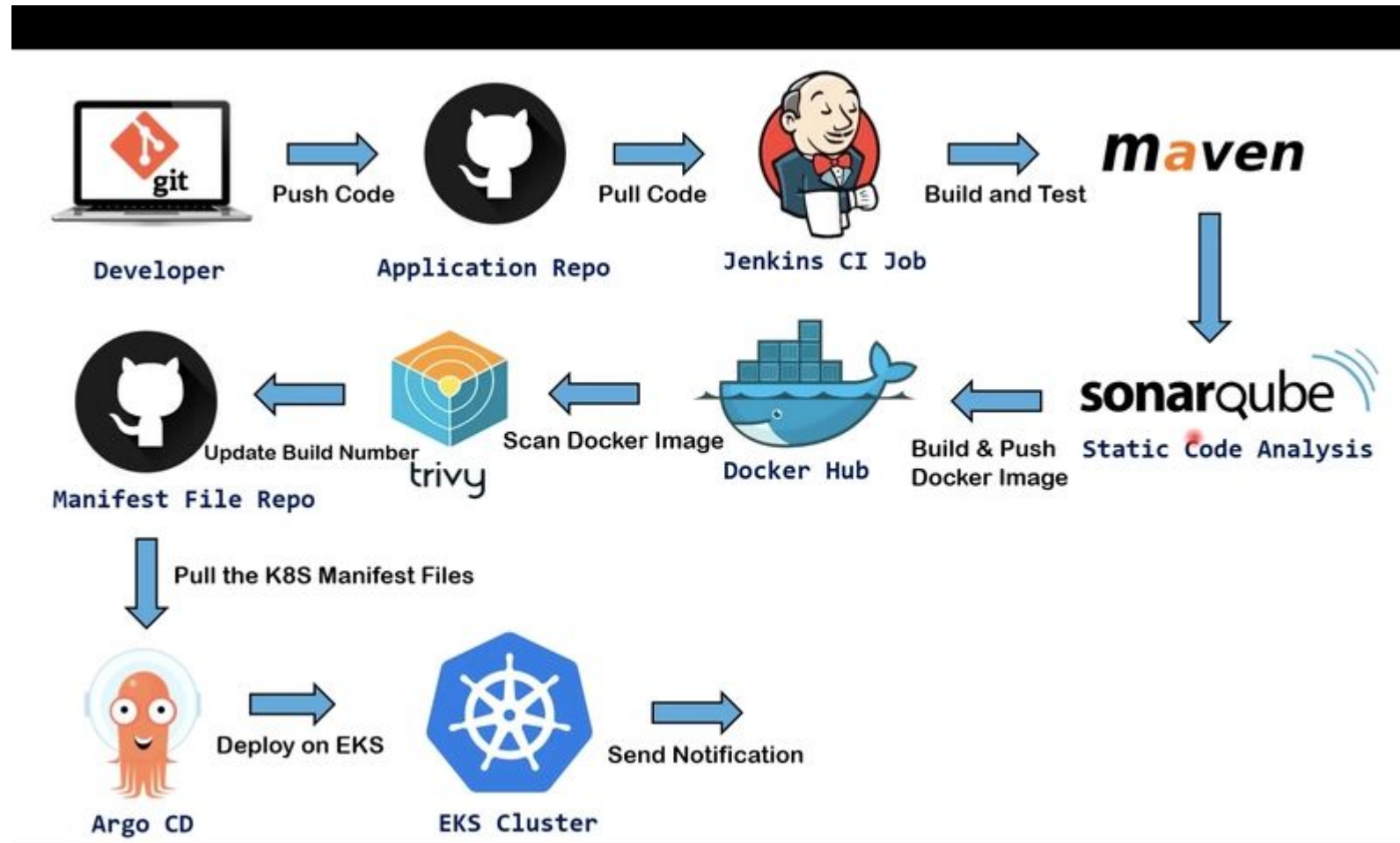


Role of Kubernetes in CD

Kubernetes automates deployment, scaling, and management of containerized applications in CD pipelines:

- Deploys containerized applications (e.g., Docker images)
- Manages clusters of machines (nodes)
- Runs applications inside pods

Kubernetes



Thank you !